

Assignment 3: End-to-End ML Versioning

Part A: Git Fundamentals & Advanced Commands

Student: Munam Repository: github.com/Munam32/MLOps Working branch: dev

Five terms used throughout

Commit	A saved snapshot of the project, with a message and an ID such as <code>a7ab962</code> .
Branch	A separate line of commits. <code>main</code> is the stable version and <code>dev</code> is where new work happens.
Staging area	The list of changes that will go into the next commit. <code>git add</code> puts changes there.
HEAD	The commit you are currently on. <code>HEAD~1</code> means "one commit before it".
Remote	The copy of the repository on GitHub, called <code>origin</code> .

A1 Create the repository

Goal: start the project on `main` with a README and a `.gitignore`.

The first commit adds two files. The `.gitignore` tells Git to ignore the virtual environment, Python cache files, DVC's cache and `.h5` model files. DVC will handle large files instead.

```
$ git show --stat <first commit>
Initial Commit
.gitignore | 4 ++++
README.md | 1 +

$ git show main:.gitignore
.venv/
__pycache__/
.dvc/cache/
*.h5
```

A2 Work on a separate branch

Goal: do all new work on `dev`, in small commits.

I created the branch with `git checkout -b dev` and made one commit per script or config file. Before the Part A tasks below, `dev` had 8 commits that `main` did not.

```
$ git log --oneline main..dev
85e40dd feat: add evaluate.py, metrics.json, confusion_matrix.png
bcbf0cc feat: create train.py
8e2f0da chore: modified params.yaml opening structure
6e3f3a6 feat: add params.yaml and preprocess.py
184b6fd chore: changing keras declaration
9901c46 feat: update .gitignore
7c201e3 feat: setup requirements.txt
7d60d03 feat: create prepare.py
```

A3 Four ways to view history

Goal: see what each `git log` option adds to the plain version.

COMMAND	WHAT IT ADDS
<code>git log --oneline --graph --all</code>	Shows every branch, one line per commit, with lines drawing where branches split and join.
<code>git log --stat -3</code>	For the last 3 commits, lists which files changed and how many lines.
<code>git log -p -1</code>	For the latest commit, shows the exact lines added (+) and removed (-).
<code>git log main..dev</code>	Lists only commits on <code>dev</code> that <code>main</code> doesn't have yet.

1 · graph of all branches

```
$ git log --oneline --graph --all
* 85e40dd feat: add evaluate.py, metrics.json, confusion_matrix.png
* bcbf0cc feat: create train.py
* 8e2f0da chore: modified params.yaml opening structure
* 6e3f3a6 feat: add params.yaml and preprocess.py
* 184b6fd chore: changing keras declaration
* 9901c46 feat: update .gitignore
* 7c201e3 feat: setup requirements.txt
* 7d60d03 feat: create prepare.py
* 33fa36b Initial Commit
```

2 · files changed per commit

```
$ git log --stat -3
commit 85e40dd3ee2c106fc2283d4ab65eefdfa9468640
Author: Munam32 <abdulmunam891@gmail.com>
Date:   Wed Sep 30 12:51:37 2026 +0500
    feat: add evaluate.py, metrics.json, confusion_matrix.png
confusion_matrix.png | Bin 0 -> 92050 bytes
metrics.json         |   4 ++++
src/evaluate.py      |  62 ++++++
3 files changed, 66 insertions(+)
... 15 more lines
```

3 · the actual changes

```
$ git log -p -1
commit 85e40dd3ee2c106fc2283d4ab65eefdfa9468640
    feat: add evaluate.py, metrics.json, confusion_matrix.png
diff --git a/confusion_matrix.png b/confusion_matrix.png
diff --git a/metrics.json b/metrics.json
+ "test_loss": 0.34686097502708435,
+ "test_accuracy": 0.8863999843597412
diff --git a/src/evaluate.py b/src/evaluate.py
```

4 · commits on dev but not main

```
$ git log main..dev
    feat: add evaluate.py, metrics.json, confusion_matrix.png
    feat: create train.py
    chore: modified params.yaml opening structure
    feat: add params.yaml and preprocess.py
    chore: changing keras declaration
    feat: update .gitignore
    feat: setup requirements.txt
    feat: create prepare.py
```

A4 Compare changes with git diff

Goal: see changes at each stage: edited, staged, and between branches.

I expanded the README. Before `git add`, `git diff` shows the edit. After `git add`, `git diff` is empty, because the change moved to the staging area. `git diff --staged` now shows it.

(a) edited, not staged yet

```
$ git status --short
M README.md

$ git diff
diff --git a/README.md b/README.md
index 3d0e341..c904c4f 100644
--- a/README.md
+++ b/README.md
@@ -1,23 @@
-## MLOps Assignment 1
\ No newline at end of file
... 23 more lines
```

(b) after git add

```
$ git add README.md
(no output)

$ git diff
(no output)

$ git diff --staged --stat
README.md | 24 ++++++
1 file changed, 23 insertions(+), 1 deletion(-)
```

Two dots vs three dots. This comparison was run after the A6 hotfix fixed the README title on `main`.

two dots

```
$ git diff main..dev -- README.md
-## MLOps Assignment 3
+## MLOps Assignment 1: Fashion-MNIST ANN Pipeline
```

three dots

```
$ git diff main...dev -- README.md
-## MLOps Assignment 1
+## MLOps Assignment 1: Fashion-MNIST ANN Pipeline
```

COMMAND	COMPARES	RESULT HERE
<code>main..dev</code>	The latest <code>main</code> with the latest <code>dev</code>	Also shows <code>main</code> 's new fix, reversed (<code>-Assignment 3</code>), as if <code>dev</code> were undoing it.
<code>main...dev</code>	The point where <code>dev</code> split off with the latest <code>dev</code>	Shows only what <code>dev</code> changed. This is what a pull request shows.

A5 Pause unfinished work with git stash

Goal: switch branches without committing a half-finished edit.

I was halfway through editing `preprocess.py` and needed to check something on `main`. `git stash` put the edit aside and left the folder clean, so I could switch branches. Back on `dev`, `git stash list` showed the saved edit, and `git stash pop` brought it back.

```

$ git status --short
M src/preprocess.py

$ git stash push -m 'wip: pixel range check in preprocess'
Saved working directory and index state On dev: wip: pixel range check in preprocess

$ git switch main
Switched to branch 'main'

$ git switch dev
Switched to branch 'dev'

$ git stash list
stash@{0}: On dev: wip: pixel range check in preprocess

$ git stash pop
    modified:   src/preprocess.py
Dropped refs/stash@{0} (45d54c2fdc7713f4f6b30d8f1cd354be66153d3e)

```

In short: stash is a temporary shelf for uncommitted changes.

A6 Hotfix, then rebase

Goal: fix something on `main`, then move `dev` on top of that fix.

1. Hotfix. `main`'s README said "Assignment 1" instead of "Assignment 3". I fixed it on a short-lived `hotfix` branch, merged that into `main`, and deleted the branch.

```

$ git switch -c hotfix
Switched to a new branch 'hotfix'

$ git commit -am 'fix: correct assignment number in README'
[hotfix 708ec94] fix: correct assignment number in README

$ git switch main
Switched to branch 'main'

$ git merge --no-ff hotfix
Merge made by the 'ort' strategy.

$ git branch -d hotfix
Deleted branch hotfix (was 708ec94).

```

2. Rebase. `git rebase main` replays each `dev` commit on top of the new `main`, as if `dev` had started from there.

3. Conflict. Both branches had changed the README's first line, so Git stopped and asked me to choose. I kept `dev`'s longer title with `main`'s corrected number, then continued.

```

Conflict

$ git rebase main
CONFLICT (content): Merge conflict in README.md

$ head -4 README.md
<<<<<<< HEAD
## MLOps Assignment 3
=====
## MLOps Assignment 1: Fashion-MNIST ANN Pipeline
... 2 more lines

```

```

Fixed

$ head -3 README.md
## MLOps Assignment 3: Fashion-MNIST ANN Pipeline
... 1 more lines

$ git add README.md
(no output)

$ git rebase --continue
Successfully rebased and updated refs/heads/dev.

```

4. Result. Before, `dev` branched off the old `main` (left). After, it sits on top of the merged hotfix (right).

Before

```
$ git log --oneline --graph --all
* dfdbc3 chore: add scratch script to inspect raw array
s
* 2640bbc feat: assert normalized pixels stay within [0,
1]
* eb24041 docs: expand README with pipeline overview and
setup
* 85e40dd feat: add evaluate.py, metrics.json, confusion
_matrix.png
* bcbf0cc feat: create train.py
* 8e2f0da chore: modified params.yaml opening structure
* 6e3f3a6 feat: add params.yaml and preprocess.py
* 184b6fd chore: changing keras declaration
* 9901c46 feat: update .gitignore
* 7c201e3 feat: setup requirements.txt
* 7d60d03 feat: create prepare.py
| * bdd6d6f Merge branch 'hotfix'
|/|
| * 708ec94 fix: correct assignment number in README
|/
* 33fa36b Initial Commit
```

After

```
$ git log --oneline --graph --all
* a7ab962 chore: add scratch script to inspect raw array
s
* 2c98562 feat: assert normalized pixels stay within [0,
1]
* 9f867b5 docs: expand README with pipeline overview and
setup
* 5879d42 feat: add evaluate.py, metrics.json, confusion
_matrix.png
* b986e3c feat: create train.py
* 6599485 chore: modified params.yaml opening structure
* 2677bd3 feat: add params.yaml and preprocess.py
* 2598fdb chore: changing keras declaration
* 5532086 feat: update .gitignore
* 51db866 feat: setup requirements.txt
* 96c2748 feat: create prepare.py
| * bdd6d6f Merge branch 'hotfix'
|/
| * 708ec94 fix: correct assignment number in README
|/
... 10 more lines
```

In short: after the rebase, dev is one straight line on top of main, and it already includes the hotfix. Rebasing gives commits new IDs, so dev had to be pushed with `git push --force-with-lease`.

A7 Undo commits with git reset

Goal: compare the gentle and the destructive way to undo a commit.

On a throwaway scratch branch, I made a commit that adds throwaway1.txt and undid it with `--soft`. Then I made a commit that adds throwaway2.txt and undid it with `--hard`.

soft: commit undone, file still staged

```
$ git reset --soft HEAD~1
(no output)

$ git log --oneline -1
a7ab962 chore: add scratch script to inspect raw arrays

$ git status --short
A throwaway1.txt
```

hard: commit and file both gone

```
$ git reset --hard HEAD~1
HEAD is now at b5e4951 chore: throwaway commit 1

$ git status --short
(no output)

$ ls throwaway2.txt
ls: cannot access 'throwaway2.txt': No such file or dire
ctory
```

COMMAND	COMMIT	YOUR CHANGES
<code>git reset --soft HEAD~1</code>	Undone	Kept and still staged (A throwaway1.txt), ready to commit again.
<code>git reset --hard HEAD~1</code>	Undone	Deleted. The file no longer exists.

A8 Move and delete files through Git

Goal: let Git record a move and a deletion directly, instead of deleting and re-adding by hand.

I moved the confusion-matrix image into a reports/ folder (and updated evaluate.py to save it there). I also removed a scratch script that was no longer needed. Each was committed separately.

git mv (R = renamed)

```
$ git mv confusion_matrix.png reports/confusion_matrix.p
ng
(no output)

$ git status --short
R confusion_matrix.png -> reports/confusion_matrix.png
```

git rm (D = deleted)

```
$ git rm scratch_inspect.py
rm 'scratch_inspect.py'

$ git status --short
D scratch_inspect.py
```

In short: `git mv` is recorded as a rename (R), so the file keeps its history. `git rm` deletes the file and stages the deletion (D) in one step.